Übung 11 – Speisekarte aus der Datenbank

1000 Datenbank · Teil 4 Website und Datenbank · ca. 90 Minuten · XAMPP, VS Code, Edge



Luigi ist genervt

„Ändert Mario einen Preis, muss jemand speisekarte.js anpassen. Die Datenbank kennt doch die Preise! Holt euch die Speisekarte von dort.“

Ziel	Die Speisekarte der Website kommt aus der Datenbank: `api/artikel.php` liefert JSON, `ladeSpeisekarte()` holt es per fetch – wie die Kategorien auf den Folien.
Startpunkt	dein Repository in `C:\xampp\htdocs\marios-pizza` (TAI12-Startpunkt) – oder der Ordner `Startpunkt`; Datenbank frisch aus `marios_pizza_start.sql`
Du schreibst	`api/artikel.php` (Aufgabe 3), `ladeSpeisekarte()` in `script.js` (Aufgabe 4)
Folien	Präsentation „1000 Datenbank“ – vor der Übung erklärt und vorgemacht; bei jeder Aufgabe steht die passende Folie

Aufgabe 1 – Umgebung prüfen

Hilfe: *Präsentation „1000 Datenbank“, Folie 0800, 0850 (Nummer unten rechts)*

1. Apache und MySQL laufen, das Projekt liegt in `C:\xampp\htdocs\marios-pizza`.
2. `api/config.php` gibt es (Kopie von `config.example.php`).
3. Rufe auf: http://localhost/marios-pizza/api/ping.php, …/api/kategorien.php und …/api/artikel.php

Aufruf	Antwort
api/ping.php	{"ok":true,"artikel":11}
api/kategorien.php (Beispiel aus den Folien)	HTTP 200. Pizza: 6 Artikel, Getränk: 3 Artikel, Dessert: 2 Artikel.
api/artikel.php	Zu Beginn HTTP 501 mit „Noch nicht programmiert“. Nach Aufgabe 3 HTTP 200 mit 11 Artikeln als JSON.
http://localhost/marios-pizza/ – Text über der Speisekarte	Zu Beginn „Ersatzdaten aus speisekarte.js …“. Nach Aufgabe 4 „Speisekarte aus der Datenbank geladen.“

Der Arbeitsbestand enthält 11 Artikel aus den bisherigen Übungen. Die Angabe 9 im Aufgabenblatt bezieht sich auf den ursprünglichen Startbestand.

Aufgabe 2 – Die Verbindung verstehen

Hilfe: *Präsentation „1000 Datenbank“, Folie 0820 (Nummer unten rechts)*

Lies `api/db.php` und beantworte:

1. Wofür steht **PDO**, und was bewirkt `PDO::ERRMODE_EXCEPTION`?

PDO bedeutet PHP Data Objects. Es stellt die Verbindung zur Datenbank her und ermöglicht SQL-Abfragen aus PHP. PDO::ERRMODE_EXCEPTION meldet Fehler als Exceptions, die ein catch-Block behandeln kann.

2. Warum bekommt der Besucher bei einem Verbindungsfehler nur eine allgemeine Meldung, während die Ursache mit `error_log()` ins Log geschrieben wird?

Die allgemeine Meldung informiert den Besucher über den Verbindungsfehler. Technische Details können interne Pfade und Verbindungsangaben enthalten. error_log() schreibt die genaue Ursache ins Serverprotokoll, damit der Entwickler den Fehler untersuchen kann.

3. Warum steht `api/config.php` in `.gitignore`? Prüfe mit `git status`.

api/config.php enthält die lokalen Zugangsdaten zur Datenbank. Der Eintrag in .gitignore schützt diese Angaben bei der Veröffentlichung. git check-ignore -v api/config.php bestätigt die Regel. Die Datei erscheint nicht in git status.

Aufgabe 3 – api/artikel.php schreiben

Hilfe: *Präsentation „1000 Datenbank“, Folie 0810, 0830 (Nummer unten rechts)*

Öffne `api/artikel.php` und löse die Schritte 1 bis 4 aus den Kommentaren – Vorbild ist `api/kategorien.php` (liegt in deinem Projekt, auf der Folie erklärt). Die Spalten müssen `id`, `name`, `beschreibung`, `preis` und `kategorie` heißen (siehe `speisekarte.js`).

Die vorhandenen Anweisungen require, die GET-Prüfung und verbinde() stellen die Verbindung bereit. Die vier Lösungsschritte für SQL, Abruf, Zahlentypen und JSON lauten wie folgt.

```
$sql = 'SELECT a.artikel_id AS id,
               a.name, a.beschreibung, a.preis,
               k.bezeichnung AS kategorie
        FROM artikel AS a
        INNER JOIN kategorie AS k
            ON a.kategorie_id = k.kategorie_id
        ORDER BY k.kategorie_id, a.artikel_id';

$zeilen = $pdo->query($sql)->fetchAll();

foreach ($zeilen as &$zeile) {
    $zeile['id'] = (int) $zeile['id'];
    $zeile['preis'] = (float) $zeile['preis'];
}
unset($zeile);

antworte(200, $zeilen);
```

Prüfergebnis im aktuellen Projekt: HTTP 200 mit 11 Artikeln. id ist eine ganze Zahl, preis ein numerischer Wert. antworte() erzeugt die JSON-Antwort über die gemeinsame Hilfsfunktion in db.php.

Aufgabe 4 – ladeSpeisekarte() mit fetch

Hilfe: *Präsentation „1000 Datenbank“, Folie 0840 (Nummer unten rechts)*

Ergänze `ladeSpeisekarte()` in `script.js` nach den Schritten im Kommentar – wie `ladeKategorien()` auf der Folie. Bei einem Fehler läuft die Seite mit den Ersatzdaten weiter. Lade die Seite: Über der Speisekarte steht „Speisekarte aus der Datenbank geladen.“

```
async function ladeSpeisekarte() {
    try {
        const antwort = await fetch("api/artikel.php");
        if (!antwort.ok) {
            throw new Error("HTTP " + antwort.status);
        }
        const daten = await antwort.json();
        speisekarte = daten;
        quelle.textContent = "Speisekarte aus der Datenbank geladen.";
    } catch (fehler) {
        quelle.textContent =
            "Datenbank nicht erreichbar – Ersatzdaten werden gezeigt.";
    }
}
```

fetch() ruft die API auf. await wartet auf die Antwort, json() liest die Daten und speisekarte erhält die Artikel. Bei einem Fehler bleibt der anfängliche Ersatzdatensatz verfügbar. Der Aufruf start() lädt zuerst die Daten und zeichnet danach die Speisekarte.

Beobachtet im frisch geladenen Browserfenster: „Speisekarte aus der Datenbank geladen.“ Beim ersten Laden war noch eine ältere JavaScript-Version im Browser aktiv.

Aufgabe 5 – Der Beweis

Hilfe: *Präsentation „1000 Datenbank“, Folie 0790 (Nummer unten rechts)*

Ändere in der Workbench einen Preis (`UPDATE artikel SET preis = 13.00 WHERE artikel_id = 1;`) und lade die Seite neu. Was passiert – und warum musst du keinen Code ändern? Danach zurück auf 12.50.

Vorbereiteter Ausgangspreis: 12,50. Der praktische Nachweis mit der Änderung auf 13,00 und der anschließenden Rücksetzung ist noch offen.

Musterlösung für den ersten Schritt in der Workbench.

```
USE marios_pizza;
UPDATE artikel
SET preis = 13.00
WHERE artikel_id = 1;
```

Erwartetes Ergebnis nach dem Neuladen: Margherita wird mit 13,00 $ angezeigt. PHP liest den aktuellen Preis aus artikel. JavaScript ruft die API erneut auf und stellt die erhaltenen Daten dar.

Nach der sichtbaren Kontrolle den Preis wieder zurücksetzen und die Seite erneut laden.

```
UPDATE artikel
SET preis = 12.50
WHERE artikel_id = 1;
```

Das kann ich jetzt

- ☐ `ping.php` liefert `{"ok":true,"artikel":9}`, `artikel.php` liefert 9 Artikel als JSON.
- ☐ Die Seite zeigt „Speisekarte aus der Datenbank geladen.“
- ☐ Eine Preisänderung in der Workbench erscheint ohne Codeänderung auf der Seite.
- ☐ Ich erkläre die Aufgaben von Frontend, Backend und Datenbank.

Zusatzaufgabe

Sortiere die Speisekarte in `artikel.php` innerhalb jeder Kategorie nach Preis.

Musterlösung für die zusätzliche Sortierung. Diese Erweiterung ist als nächster eigener Arbeitsschritt vorgesehen.

```
ORDER BY k.kategorie_id, a.preis ASC, a.artikel_id
```

kategorie_id hält die Kategorien zusammen. preis ASC ordnet die Artikel jeder Kategorie vom günstigsten zum teuersten. artikel_id legt die Reihenfolge bei gleichen Preisen fest.

Zum Schluss: sichern mit Git

Prüfe zuerst mit `git status`, dass `api/config.php` **nicht** in der Liste steht. Dann:

```
git add .
git commit -m "Backend: Speisekarte aus der Datenbank (Übung 11)"
git push
```

Kontrolle: auf github.com das Repository neu laden – dein Commit und die neuen Dateien sind zu sehen.